



Spring Boot REST API

1. REST API basics
2. Setting up an application
3. Developing different layers
4. Custom operations
5. Testing with postman

- REST is a standardized approach to building web services.
- The REST architecture builds web services that are accessible through URLs.
- REST API is accessible using HTTP protocol for communication.
- REST is mainly producing and consuming JSON data.
- The most commonly used REST methods are : POST, GET, PUT, and DELETE.

- Creating application:
 - Create a spring boot application by adding the dependency for spring-web, spring-data-jpa and mysql-driver. Optional dependency like spring devtools can be very helpful.
- Setting application database
 - Spring Data JPA, which allows us to set this connection up with just a couple of parameters.
 - We need to set few properties in application.properties file which is a information of database connectivity.

- Spring Boot REST APIs typically follow a layered architecture for better organization
- Presentation layer
 - This is the outermost layer which communicates with the client.
 - Class with `@RestController` and methods with appropriate mapping annotations.
 - This layer converts JSON data into objects and vice-versa
- Business layer
 - Responsible for handling the core business logic
 - These are POJO classes annotated with `@Service`.
 - This layer is called by controllers and communicates with persistence layer
 - It can handle tasks like clubbing data meaningfully from 2 repositories, performing computations, managing transactions etc

- Persistence layer
 - This layer handles data persistence and retrieval and update operations from a database.
 - It is the interface extending from CrudRepository or JpaRepository.
 - Repositories provide built in methods and custom methods can be added with specified queries
- Entity layer
 - This layer is responsible for mapping database tables into java classes(entities).
 - It refers the class with @Table annotation and all properties with @Column annotation with one special property for primary key column annotated with @Id.

- In repository interface, we can create custom methods for retrieving specific data and for updating some records in database.
- JPQL (Java Persistence Query Language)
 - It works with entity classes and properties rather than tables and columns
 - A typical JPQL: SELECT [selection] FROM [entity name] [WHERE clause] [ORDER BY clause]
 - All clauses of SQL are valid in JPQL
 - JPQL can be written with parameters
- Custom methods
 - These are additional abstract methods which are annotated with @Query where JPQL is specified

- Method naming conventions:
 - We can create custom methods which deal with database without writing any query.
 - Spring Data JPA will create SQL queries based on method name.
 - Examples :
 1. `findByLastnameAndFirstname` - where `x.lastname = ?1` and `x.firstname = ?2`
 2. `findByAgeNotIn(Collection<Age> ages)` - where `x.age` not in `?1`
- Native SQL queries
 - Annotate your method with `@Query` and specify the native SQL query along with the `nativeQuery` attribute set to true.

- JPQL for update operation:
 - Additional `@Modifying` annotation can be used on the custom methods for DML operations along with `@Query`
- Executing procedures:
 - `@NamedStoredProcedureQuery` and `@Procedure` annotation can be used for executing stored procedures in the database
- Handling Date columns:
 - Spring Data JPA supports mapping classes like `java.util.Date`, `java.sql.Date`, `LocalDate` and `LocalDateTime` (Java 8 and above) for mapping Date properties.
 - `@Temporal` annotation for mapping Date and time properties.

- Postman is a versatile tool for testing APIs
- Core feature:
 - Request builders
 - Response viewers
- Setting Up Postman for Spring Boot Testing
 - This tool can be downloaded freely and installed.
 - New collection can be created
 - New requests can be added in the collection
- Configuring the Request
 - Minimum information that needs to be configured is request method and request URL.
 - Other parameters like headers, body can be specified as per need

- Sending the request using send button
- Viewing the response:
 - Response can be verified in the response view for status code, body and headers if any